Tutorial 3: Model-Free Monte Carlo Methods

Reinforcement Learning Course

Learning Objectives

By the end of this tutorial, you should be able to:

- Understand the difference between model-free and model-based methods
- Compute returns and value estimates using Monte Carlo methods
- Apply first-visit and every-visit MC policy evaluation
- Implement on-policy Monte Carlo control with ϵ-greedy policies
- Understand the exploration-exploitation trade-off

Notation Reference

- G_t: Return from time t, $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$
- $V^\pi(s)$: State value function under policy π
- $Q^\pi(s, a)$: Action value function under policy π
- γ: Discount factor, $0 \leq \gamma < 1$
- ϵ: Exploration parameter for ϵ-greedy policies

1 Understanding Monte Carlo Methods

1.1 MC vs DP: Conceptual Understanding

Consider the following statements about Monte Carlo (MC) and Dynamic Programming (DP) methods. For each statement, indicate whether it applies to MC, DP, both, or neither.

(a) Requires complete knowledge of transition probabilities $P(s'|s, a)$

(b) Can only be applied to episodic tasks

(c) Uses bootstrapping (updates based on other value estimates)

(d) Learns directly from experience without a model

(e) Converges to optimal policy under appropriate conditions

1.2 Computing Returns

An agent follows a trajectory and observes the following rewards: $R_1 = -1$, $R_2 = -1$, $R_3 = -1$, $R_4 = 10$ (terminal state reached).

(a) Calculate G_0 (return from time 0) with $\gamma = 0.9$

(b) Calculate G_1 (return from time 1) with $\gamma = 0.9$

(c) Calculate G_2 (return from time 2) with $\gamma = 0.9$

(d) Calculate G_0 with $\gamma = 1.0$ (no discounting)

2 Monte Carlo Policy Evaluation

2.1 First-visit vs Every-visit MC

Consider a simple 3-state MDP with states $\{s_1, s_2, s_3\}$. An agent following policy π generates the following three episodes (with $\gamma = 0.9$):

Episode 1: $s_1 \xrightarrow{r=2} s_2 \xrightarrow{r=-1} s_1 \xrightarrow{r=1} s_2 \xrightarrow{r=0} s_3 \xrightarrow{r=10}$ (terminal)

Episode 2: $s_2 \xrightarrow{r=3} s_1 \xrightarrow{r=-2} s_3 \xrightarrow{r=10}$ (terminal)

Episode 3: $s_1 \xrightarrow{r=1} s_2 \xrightarrow{r=2} s_1 \xrightarrow{r=-1} s_2 \xrightarrow{r=1} s_1 \xrightarrow{r=0} s_3 \xrightarrow{r=10}$ (terminal)

Initialize: $V(s_1) = 0$, $V(s_2) = 0$, $V(s_3) = 0$

(a) Compute the returns for each visit to s_1 across all episodes

(b) Compute the returns for each visit to s_2 across all episodes

(c) Calculate $V(s_1)$ after these three episodes using **first-visit** MC

(d) Calculate $V(s_1)$ after these three episodes using **every-visit** MC

(e) Calculate $V(s_2)$ after these three episodes using **first-visit** MC

(f) Calculate $V(s_2)$ after these three episodes using **every-visit** MC

3 Programming Exercises

3.1 Implementing Monte Carlo Policy Evaluation

Open `tutorial_template.py`. Your task is to implement Monte Carlo policy evaluation for a given policy in GridWorld.

(a) **Implement** `generate_episode`: Generate an episode by following a given policy until reaching a terminal state.

(b) **Implement** `first_visit_mc_evaluation`: Estimate $Q^\pi(s, a)$ using first-visit Monte Carlo. The algorithm should:

  (a) Generate episodes following the policy

  (b) For each state-action pair's first visit in an episode, compute the return

(c) Average returns across episodes

(d) Converge when estimates stabilize

(c) **Test your implementation**: Run the provided test function that evaluates a simple policy (go right/down) on a simple maze.

Expected output:

- Your code should generate `mc_evaluation.png`
- Q-values near the goal should be higher (greener arrows)
- Q-values for actions leading toward the goal should be higher than those leading away
- Convergence should occur within 5000 episodes for the simple maze

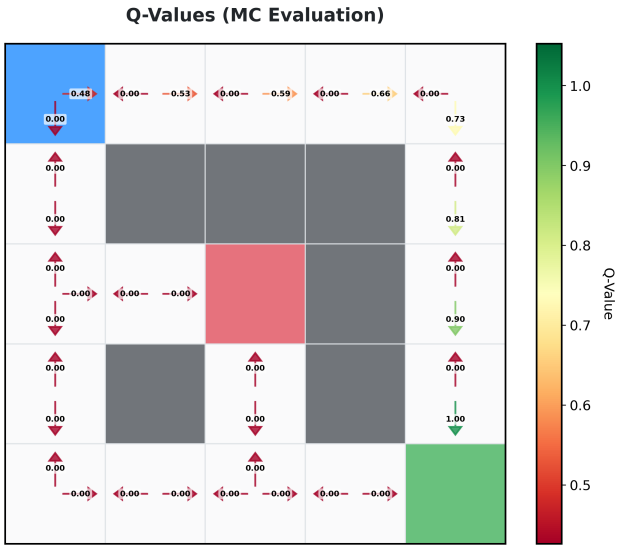


Figure 1: Q-values from evaluating a right/down policy. Green arrows indicate high Q-values, red arrows indicate low Q-values. Notice how RIGHT and DOWN actions near the goal have higher values. **NOTE: You may not get this exact result (this is Monte Carlo after all!), but your values should show similar patterns.**

3.2 Implementing On-policy MC Control

Extend your code to implement on-policy Monte Carlo control with ϵ-greedy exploration.

(a) **Implement** `epsilon_greedy_policy`: Given Q-values and ϵ, return an ϵ-greedy policy.

(b) **Implement** `on_policy_mc_control`:

(a) Initialize Q-values and policy

(b) Generate episodes using current ϵ-greedy policy

(c) Update Q-values based on returns

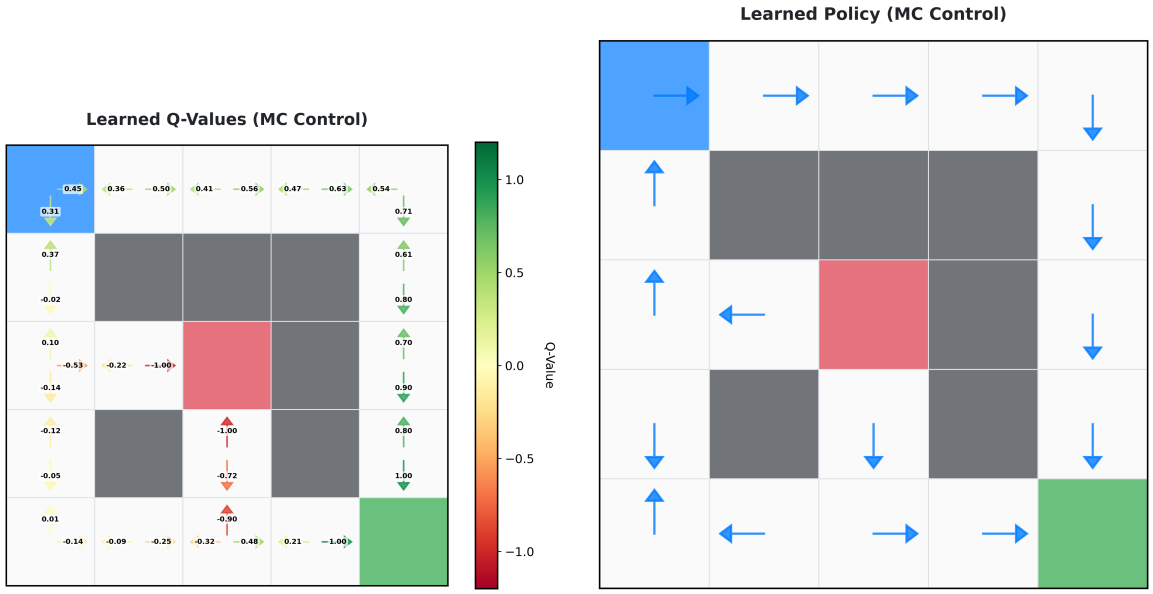
(d) Update policy to be ϵ-greedy with respect to new Q-values

(e) Repeat until convergence

(c) **Implement ϵ decay**: Start with $\epsilon = 1.0$ and decay to $\epsilon_{\text{min}} = 0.1$ to gradually shift from exploration to exploitation.

Expected output:

- Your code should generate `mc_control_q_values.png` and `mc_control_policy.png`
- Initial episodes should be highly exploratory (random-like)
- Final policy should mostly take optimal actions (with some exploration)
- Convergence within 10,000 episodes for simple maze



(a) Learned Q-values (b) Extracted optimal policy

Figure 2: Results from on-policy MC control. (a) Shows all Q-values with green indicating high values - notice how actions pointing toward the goal are greener. (b) Shows the extracted greedy policy - arrows point toward the goal while avoiding traps and walls. **NOTE: You may not get this exact result (this is Monte Carlo after all!), but your values should show similar patterns.**